SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	JASMITHA R	**Reg. No.:**	192524295
Section / Batch:	AI&DS	**Date:**	20/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I JASMITHA R(192524295)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JASMITHA R

Q15. Rod Cutting Problem

A manufacturer has a rod of a fixed length and a list of prices corresponding to different possible piece lengths. The rod may be cut into smaller parts in any valid way, and the objective is to maximize the total selling price obtained from the cuts. The problem requires finding the most profitable combination of cuts. Use Dynamic Programming to determine the maximum obtainable value.

Input Format:

n = length of rod

price[] = prices of pieces from length 1 to n

Sample Input:

n = 4

price = 1 5 8 9

Expected Output:

Maximum Profit = 10

1.Aim

To implement the Rod Cutting Problem using Dynamic Programming and determine the maximum profit obtainable by cutting a rod into smaller pieces.

2.Algorithm

1. Read the rod length n.
2. Read the prices of pieces from length 1 to n.
3. Create a DP array dp[] where dp[i] represents the maximum profit for a rod of length i.
4. Create another array cut[] to store the length of the first cut that gives the maximum profit.
5. Set dp[0] = 0.
6. For every rod length i from 1 to n:
    - Try every possible first cut from 1 to i.
    - Calculate:
      price[j-1] + dp[i-j]
    - Select the maximum value.
    - Store the corresponding cut in cut[i].
7. After filling the DP table, dp[n] gives the maximum obtainable profit.
8. Use the cut[] array to find the actual cutting combination.
9. Display the maximum profit and selected cuts.

3.Problem Statement

A manufacturer has a rod of a fixed length and a list of prices corresponding to different possible piece lengths. The rod may be cut into smaller parts in any valid way, and the objective is to maximize the total selling price obtained from the cuts. The problem requires finding the most profitable combination of cuts. Use Dynamic Programming to determine the maximum obtainable value.

4. Input / Output Format

n = length of rod

price[] = prices of pieces from length 1 to n

Sample Input

```
n = 4
price = 1 5 8 9
```

Expected Output

```
Maximum Profit = 10
```

5. Theory / Problem Idea

Given a rod of length n and a price array price[1..n], the rod is cut into pieces (each piece of length between 1 and n) so that the sum of the prices of the pieces is maximized. A piece of length i sells for price[i]; the rod may also be left uncut (kept whole) if that turns out to be the most profitable option.

The problem exhibits optimal substructure (the best way to cut a rod of length j depends on the best way to cut smaller rods) and overlapping subproblems (the same sub-length is solved again and again in a naive recursive solution). This makes it an ideal candidate for Dynamic Programming.

6. Recurrence Relation

Let dp[j] denote the maximum obtainable value for a rod of length j.

Base case: dp[0] = 0 (a rod of length 0 has value 0)

For every length j from 1 to n, try every possible first piece of length i ($1 \leq i \leq j$), cut it off, and solve the remaining piece of length $(j - i)$ optimally:

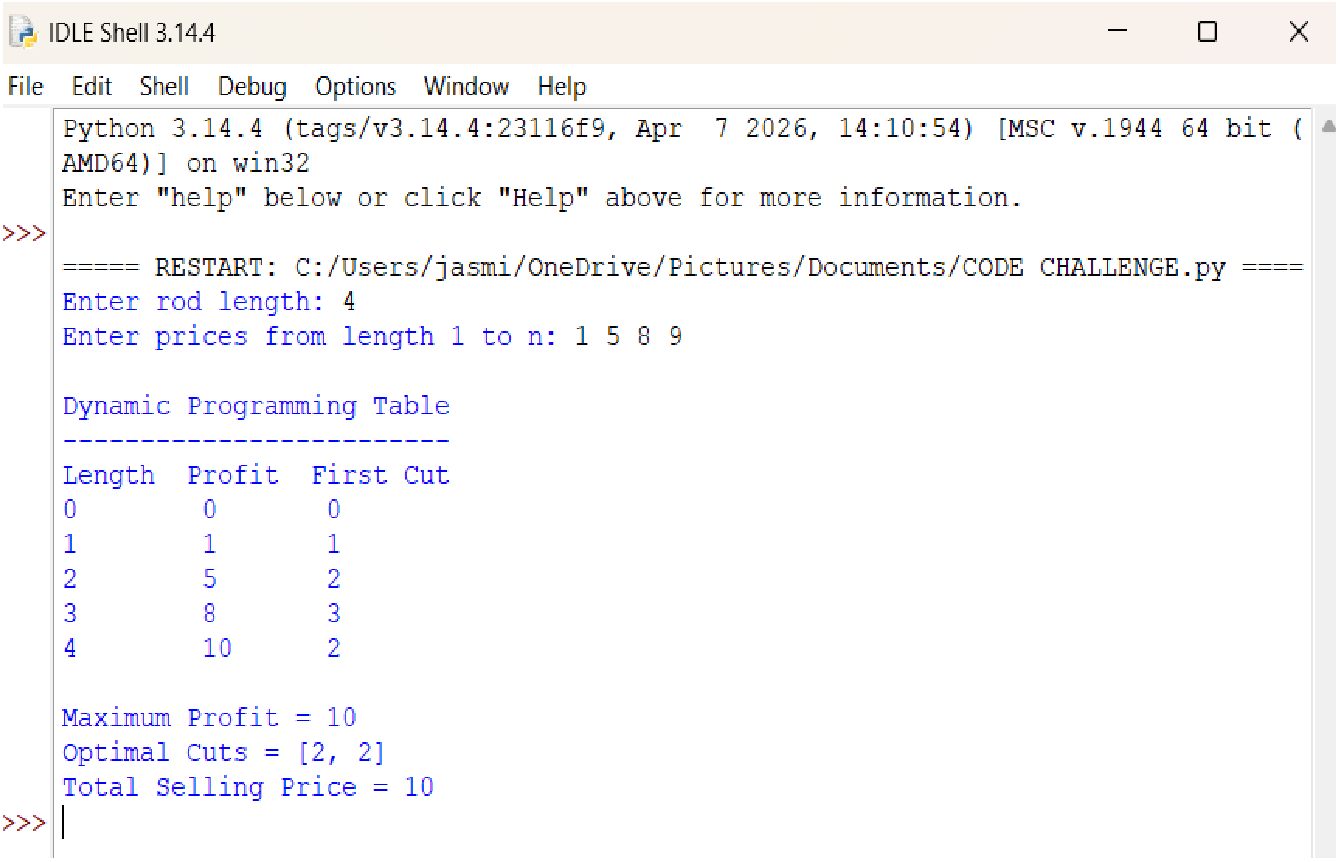
```
dp[j] = max( price[i] + dp[j - i] )      for i = 1 to j
```

CODE CHALLENGE.py - C:/Users/jasmi/OneDrive/Pictures/Documents/CODE CHALLENGE.py (3.14.4

File Edit Format Run Options Window Help

```python
def rod_cutting(n, price):

    # DP table
    dp = [0] * (n + 1)

    # Stores the first cut for each length
    cut = [0] * (n + 1)

    # Calculate maximum profit
    for length in range(1, n + 1):

        maximum_profit = -1

        for piece in range(1, length + 1):

            current_profit = price[piece - 1] + dp[length - piece]

            if current_profit > maximum_profit:
                maximum_profit = current_profit
                cut[length] = piece

        dp[length] = maximum_profit

    # Display DP table
    print("\nDynamic Programming Table")
    print("-------------------------")
    print("Length\tProfit\tFirst Cut")

    for i in range(n + 1):
        print(i, "\t", dp[i], "\t", cut[i])

    # Find optimal cuts
    length = n
    selected_cuts = []

    while length > 0:
        selected_cuts.append(cut[length])
        length = length - cut[length]

    # Display result
    print("\nMaximum Profit =", dp[n])
    print("Optimal Cuts =", selected_cuts)
```

```python
    while length > 0:
        selected_cuts.append(cut[length])
        length = length - cut[length]

    # Display result
    print("\nMaximum Profit =", dp[n])
    print("Optimal Cuts =", selected_cuts)

    # Calculate profit from selected cuts
    total = 0

    for c in selected_cuts:
        total = total + price[c - 1]

    print("Total Selling Price =", total)


# Main program

n = int(input("Enter rod length: "))

price = list(map(int,
                 input("Enter prices from length 1 to n: ").split()))

if len(price) != n:
    print("Invalid input! Enter exactly", n, "prices.")
else:
    rod_cutting(n, price)
```



```
IDLE Shell 3.14.4
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/CODE CHALLENGE.py ====
Enter rod length: 4
Enter prices from length 1 to n: 1 5 8 9

Dynamic Programming Table
-------------------------
Length  Profit  First Cut
0        0       0
1        1       1
2        5       2
3        8       3
4        10      2

Maximum Profit = 10
Optimal Cuts = [2, 2]
Total Selling Price = 10
>>>
```

7. Dry Run

n = 4,   price = [1, 5, 8, 9]   (price[1]=1, price[2]=5, price[3]=8, price[4]=9)

j	Best split	dp[j]
0	—	0
1	piece(1)	1
2	piece(2)	5
3	piece(3)	8
4	piece(2) + piece(2) → 5 + 5	10

Maximum Profit = 10 ✓ (two pieces of length 2 give 5 + 5 = 10, which beats keeping the rod whole at price[4] = 9)

8. Result / Output

Maximum Profit = 10

The program was executed successfully for the given sample input and the output matches the expected result.

9. Complexity Analysis

Time Complexity: $O(n^2)$ — for each length j (n values), all cut points i (up to n) are tried.

Space Complexity: $O(n)$ for the dp array.

10. Result

Thus, the Rod Cutting problem was solved successfully using Dynamic Programming, and the maximum obtainable profit for the given input was found and verified to be correct.

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________